Team Work Division: Secure-FEPRH Project

Based on the architecture of the **Secure-FEPRH (Faculty of Engineering Project & Research Hub)**, the system encompasses advanced backend security, real-time collaboration, artificial intelligence, and a modern React frontend.

To ensure fair distribution while assigning the most technically demanding architectural and security components to **Student 1 (You)**, the work is divided into four distinct tracks.

👨‍💻 Student 1: Core Architecture & Security (The "Hard" Part - YOU)

Focus: System Foundation, Cryptography, Multi-Tenancy, and DevOps

As the core architect, you will handle the most critical and complex infrastructure that the rest of the application relies upon. This requires deep knowledge of backend systems, security protocols, and database management.

Responsibilities:

1. **Database Architecture & Multi-Tenancy:**
2. Design the SQLAlchemy database models (`app/models.py`).
3. Implement the hierarchical data structure (Ministry → University → Faculty → Department).
4. Manage database migrations using Alembic (`alembic/`, `alembic.ini`).
5. **Authentication & Hierarchical RBAC (Role-Based Access Control):**
6. Implement secure JWT generation and validation (`app/auth.py`).
7. Build strict dependency injection bouncers in FastAPI (`app/dependencies.py`) to ensure users can only access data within their clearance level (e.g., a Dean can only see their Faculty's projects).
8. **Secure Document Repository:**
9. Implement AES-256-GCM encryption/decryption for all uploaded files (`app/crypto.py`).

10. Build the `file_validator.py` to perform magic byte analysis, blocking malicious executables before they hit the database.
11. **DevOps & Asynchronous Processing:**
12. Configure the Celery and Redis worker pipeline (`app/worker/celery_app.py`) to offload heavy background tasks (like nationwide plagiarism scanning).
13. Handle server deployment and local environment configurations (`run.py`, `.env`).

🤖 Student 2: Artificial Intelligence & Biometrics Engine

Focus: Computer Vision, Natural Language Processing, and Python Data Science

This student will focus exclusively on the AI micro-services that give the application its cutting-edge "smart" capabilities.

Responsibilities:

1. **National Plagiarism Engine:**
2. **Code Similarity (`app/ai/ast_similarity.py`):** Write algorithms to parse Python/JS code into Abstract Syntax Trees (AST), strip out variable names, and calculate structural Levenshtein distances to catch students who just rename variables.
3. **Text Similarity (`app/ai/nlp_similarity.py`):** Implement NLP pipelines using TF-IDF and Cosine Similarity to compare abstracts and project descriptions. Ensure the pipeline handles Arabic text normalization (stripping tashkeel/diacritics).
4. **Face Recognition Attendance (`app/ai/face_engine.py`):**
5. Integrate OpenCV and face recognition libraries.
6. Implement logic to convert webcam feeds into 128-dimensional facial encodings.
7. Write the Euclidean distance comparison algorithm to verify student identities against their database encodings, eliminating "buddy punching."
8. **AI Backend Routes:**
9. Expose the AI engines to the frontend via FastAPI endpoints (e.g., `/plagiarism/scan` and `/attendance/verify`).

⚡ Student 3: Real-Time Collaboration & Analytics

Focus: WebSockets, Dynamic Dashboards, and Agile Workflows

This student bridges the gap between the frontend and backend, focusing on real-time data flow, complex SQL aggregations, and interactive UI components.

Responsibilities:

1. **Real-Time Kanban Board:**
2. **Backend:** Implement WebSocket connections (`app/ws.py`, `app/routers/kanban.py`) to broadcast task movements (e.g., "To Do" → "In Progress") to all connected clients instantly.
3. **Frontend:** Build the interactive drag-and-drop Kanban Board UI (`KanbanBoard.jsx`) using libraries like `@hello-pangea/dnd`.
4. **Hierarchical Analytics Engine:**
5. **Backend:** Write complex SQLAlchemy `JOIN` queries in `app/routers/analytics.py` that dynamically calculate statistics (total projects, system health, active users) based on the requesting user's hierarchical role.
6. **Frontend:** Build the Command Center Dashboard (`DashboardHome.jsx`), integrating charting libraries to display the analytics visually.
7. **Meeting Management:**
8. Build the CRUD operations and frontend interface (`MeetingManagement.jsx`) for scheduling supervisor-student meetings and linking them to the AI attendance logs.

🎨 Student 4: Frontend Foundation, UX/UI, & User Management

Focus: React/Vite Infrastructure, Styling, State Management, and Core Portals

This student ensures the application looks professional, functions smoothly across devices, and provides intuitive portals for all user types.

Responsibilities:

1. **Global Design System & Theming:**
2. Configure Tailwind CSS (`tailwind.config.js`) and global CSS variables (`index.css`) to establish the "National Command Center" aesthetic (dark modes, glassmorphism, primary color tokens).
3. Implement global layouts (`DashboardLayout.jsx` , `Sidebar` , `Topbar`) and route protection (`App.jsx`).
4. **Authentication Flow UI:**
5. Build the login and registration pages (`Login.jsx` , `Register.jsx`) with beautiful micro-animations using Framer Motion.
6. Manage global frontend state for user sessions via React Context (`AuthContext.jsx`) and Axios interceptors (`api.js`).
7. **User Management & Profiles:**
8. Build the Admin-level User Management interface (`UserManagement.jsx`) for creating and managing university staff and students.
9. Develop the User Profile page (`UserProfile.jsx`) handling secure password changes and avatar/face-encoding uploads.
10. **Project Directory:**
11. Build the main Projects List view (`ProjectsList.jsx`) with advanced filtering, searching, and pagination capabilities.
12. Create the Project Overview portal (`ProjectDetail.jsx`).